

OmniSpec: Speculative Evidence Orchestration for Efficient Omni-Modal Agent

Yang-Hao Zhou¹, Xiao Yang^{1*}

¹Canva Research

Corresponding author: Xiao Yang

Abstract

Orchestra-based omnimodal agents coordinate modality-aware subagents to solve tasks grounded in web-scale multimodal sources. While orchestration parallelizes independent subtasks, evidence acquisition within each SubAgent still follows a sequential search–reason loop, placing web retrieval on the critical path of both latency and external API cost. We introduce OmniSpec, a speculative evidence orchestration framework motivated by the observation that an agent’s future search needs are often foreshadowed by its current multimodal reasoning state. OmniSpec couples two mechanisms: Speculative Reflection generates candidate queries from these multimodal cues and issues them asynchronously, overlapping retrieval with ongoing reasoning, while a task-wide Shared Evidence Pool stores prefetched evidence and serves subsequent search requests after normalized exact-match verification. This enables evidence reuse across modalities, reasoning steps, and SubAgents, reducing exposed search latency and redundant tool calls. On OmniGAIA, OmniSpec delivers a $5.4\times$ wall-clock speedup and reduces billable search calls and total execution cost to $0.09\times$ and $0.27\times$, respectively, while maintaining accuracy comparable to the grounded-search orchestra baseline. OmniSpec further yields $1.7\text{--}3.9\times$ end-to-end speedups on MM-BrowseComp-Acad. and Video-BrowserComp. These results demonstrate that evidence-level speculation can effectively accelerate orchestra-based omnimodal agents grounded in external evidence.

Introduction

Omni large language models are rapidly evolving from passive perceivers of text, image, audio, and video into agents that actively seek, verify, and synthesize external evidence (Li et al. 2026; Xing et al. 2026). Recent orchestra-style systems scale such agents through task decomposition: a MainAgent splits the query into modality-aware subtasks, delegates them to specialized SubAgents, and aggregates their findings (Lin et al. 2025; Zhang et al. 2026). This design parallelizes independent reasoning branches—but the parallelism stops at the subtask boundary. Inside each SubAgent, evidence acquisition still follows a ReAct-style loop: reason, issue a web search, wait, update, and formulate the next query. Every search therefore sits on the critical path of both latency and external API cost (Figure 1).

We call this hidden bottleneck *retrieval depth*: the number of sequentially dependent retrieval actions on a Sub-

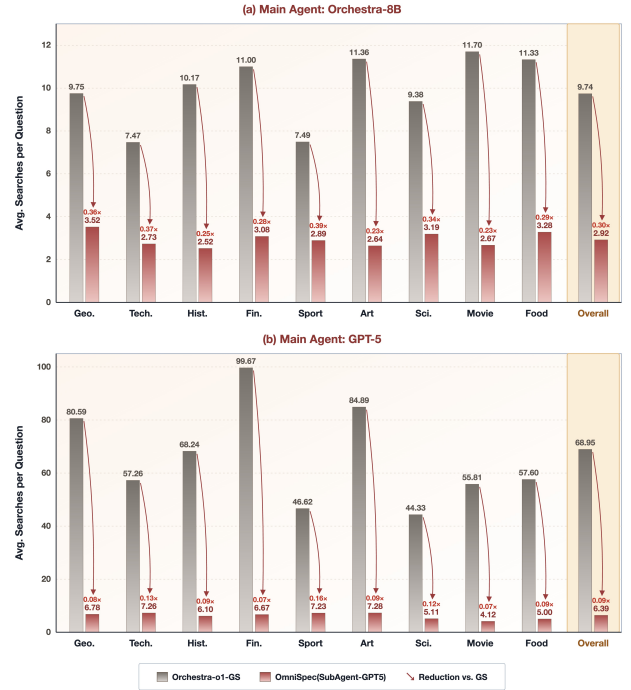


Figure 1: Average billable search calls per question on OmniGAIA (Li et al. 2026), comparing OmniSpec (GPT-5 SubAgent) with the Orchestra-o1-GS baseline under (a) Orchestra-o1-8B and (b) GPT-5 MainAgents. OmniSpec cuts search volume to $0.30\times$ and $0.09\times$ overall, respectively, with uniform reductions across all nine categories.

Agent’s critical path. Each dependent search adds exposed waiting time, and each reformulated query consumes billed API budget. In practice, search-heavy tasks exhibit *search thrashing*—an entity, its variant, a disambiguated form, and a verification query issued as separate serial calls for what is essentially one information need. Omnimodal inputs make this worse: before an entity mentioned in an audio clip is bound to its speaker, or an object in a video frame to a concrete name and timestamp, the SubAgent issues premature generic queries over ambiguous surface forms, each retrieving weakly relevant evidence and triggering further reformu-

*Corresponding author.

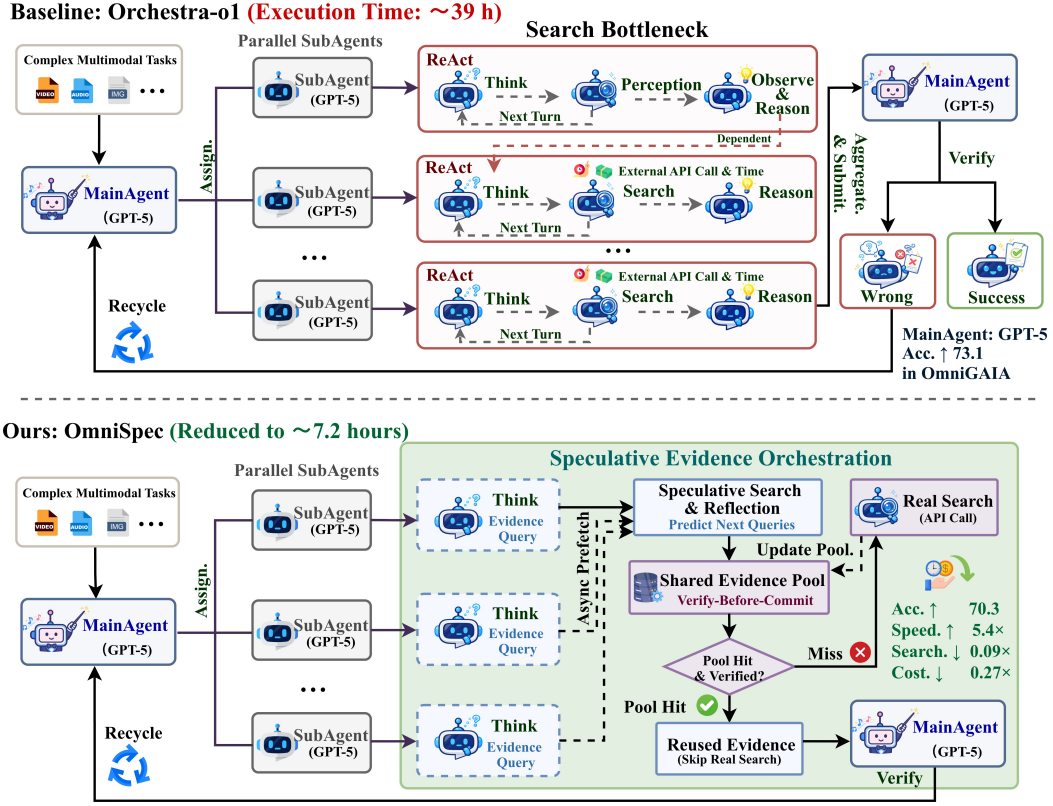


Figure 2: Overview of OMNISPEC. **Top:** in the Orchestra-style baseline, each SubAgent acquires web evidence through a serial think–search–verify loop (red dashed boxes); failed verification triggers further dependent searches, placing every retrieval on the latency- and cost-critical path. **Bottom:** OmniSpec inserts a speculative layer at the tool level: predicted queries are prefetched asynchronously into a shared, task-level Evidence Pool, and an authoritative SubAgent query is served from the pool only under verified normalized match (*Pool Hit & Verified?*), skipping the external search; otherwise the system falls back to real search. The orchestrator–SubAgent interface is unchanged.

lation. Cross-modal ambiguity thus directly inflates retrieval depth.

Existing acceleration techniques do not shorten this path. Token-level speculative decoding accelerates generation but leaves the tool-use trajectory unchanged (Leviathan, Kalman, and Matias 2023; Chen et al. 2023). Action- and tool-level speculation executes likely future actions early, but succeeds mainly when the next action is structurally recoverable from the previous observation—such as visiting a URL returned by search (Ye et al. 2025; Sui et al. 2026). A search query is different: it is *synthesized* from partial evidence, uncertainty, and evolving task intent, so exact next-query prediction is brittle precisely where search dominates latency and cost.

Our key observation is that web search is safe to speculate on in a way that most agent actions are not: it is read-only, side-effect-free, and embarrassingly parallel. Several plausible queries can be issued concurrently, and unused results simply discarded. This changes the objective of speculation—the system need not predict the single next query, only *cover* likely future evidence needs cheaply enough that hits offset misses. We call this reformulation *evidence-level speculation*: covering evidence needs rather

than predicting actions.

We instantiate this idea in OmniSpec, a speculative evidence orchestration framework for budget-aware omnimodal agents (Figure 2). During SubAgent execution, a lightweight predictor infers likely next search intents from the question, recent observations, and the accumulated evidence state. A *speculative reflection* step grounds candidate queries in cross-modal cues, binding ambiguous entities to the speakers, objects, or timestamps observed in audio, image, and video inputs. OmniSpec then issues a bounded set of grounded candidates asynchronously while the SubAgent continues reasoning, storing results in a task-wide, shared *Evidence Pool*. When the SubAgent later emits an authoritative query, the cached result is reused only under verified normalized match; otherwise the agent searches as usual. Speculation thus affects only *when* a query is executed, never *what* the SubAgent asks: the original reasoning trajectory and agent interface are preserved, while the pool amortizes retrieval across modalities, steps, and SubAgents. Fan-out, concurrency, and pool capacity are all explicitly budgeted, so speculation improves evidence economics rather than trading cost for latency.

We integrate OmniSpec into an Orchestra-style agent and evaluate it on three retrieval-heavy benchmarks. With a GPT-5 MainAgent on OmniGAIA, OmniSpec reaches 70.3% accuracy—within 2.8 points of the matched grounded-search baseline—while reducing agent-initiated search calls to $0.09\times$, total execution cost to $0.27\times$, and delivering a $5.4\times$ wall-clock speedup; with the lightweight Orchestra-o1-8B MainAgent, it retains comparable accuracy at $0.30\times$ search and $0.39\times$ cost. Consistent gains hold on Video BrowserComp and MM-BrowseComp-Acad. Trajectory-level analysis shows these savings come from collapsing the exposed retrieval critical path and reusing pooled evidence, not from stronger underlying models. Efficiency in omnimodal orchestration, we argue, should be measured not only by how fast agents reason, but by how intelligently they acquire, share, and reuse evidence.

In summary, our contributions are:

- We identify *retrieval depth* as a hidden source of sequential latency and API cost in orchestrated omnimodal agents: orchestration parallelizes across subtasks but not within them, and cross-modal entity ambiguity further deepens each SubAgent’s serial search-and-reason chain.
- We propose *evidence-level speculation*, which exploits the read-only, parallelizable nature of web search to cover likely future evidence needs through budgeted asynchronous prefetching, and commits reused evidence only under verified normalized match with the authoritative query—preserving the original reasoning trajectory and agent interface.
- We realize this design in OmniSpec, combining cross-modal speculative reflection, bounded query fan-out, verify-before-commit reuse, and a task-wide Evidence Pool. On OmniGAIA, OmniSpec stays within 2.8 points of the grounded-search baseline while cutting search calls to $0.09\times$ and execution cost to $0.27\times$ with a $5.4\times$ speedup, with consistent gains on Video BrowserComp and MM-BrowseComp-Acad.

Related Work

Agentic Omni Large Language Models

Recent multimodal systems have shifted from vision-language assistants toward *omnimodal agents* that process text, images, audio, video, and external tools. OmniGAIA introduces a benchmark for native omnimodal agents requiring deep cross-modal reasoning and multi-turn tool execution, together with OmniAtlas as a tool-integrated agent (Li et al. 2026). Subsequent systems study active perception and tool invocation (Tao et al. 2025; Xing et al. 2026), coordination of modality-specific foundation models (Lin et al. 2025), and retrieval-augmented planning for long audio-video QA (Zhu et al. 2026). Orchestra-o1 further introduces modality-aware task decomposition, online SubAgent specialization, and parallel subtask execution (Zhang et al. 2026). These works improve perception, coordination, and subtask-level parallelism, but evidence acquisition *inside* each SubAgent still follows a serial search-and-reason loop—their retrieval depth is untouched. OmniSpec targets

this complementary bottleneck, making web retrieval speculative, shared, and trajectory-preserving without modifying the orchestration policy.

Speculative Agentic Workflows

Speculative execution has progressed from token-level decoding (Leviathan, Kalman, and Matias 2023; Chen et al. 2023) to agentic systems that overlap computation across otherwise sequential decision steps (Nichols et al. 2025; Hooper et al. 2026). One class of methods predicts concrete next actions and executes them in parallel, relying on verification-based acceptance (Ye et al. 2025) or recurring tool-call patterns (Sui et al. 2026). These methods are effective when the next action is strongly implied by the current execution state. Web search poses a harder prediction problem: future queries are synthesized from an evolving belief state and are often not directly recoverable from the immediately preceding observation, making exact next-action prediction fragile in retrieval-intensive workflows. A second class speculates over plans or execution trajectories (Choi et al. 2026; Guan et al. 2025; Huang et al. 2026). Although these methods can reduce planning or perception overhead, they leave chains of sequentially dependent web searches largely on the critical path. A closely related direction, Predictive Prefetching for RAG, anticipates future retrieval requests within a single generation stream (Zhang and Pei 2026). OmniSpec instead speculates over a set of plausible evidence needs and stores prefetched results in a task-level Evidence Pool shared across reasoning steps and SubAgents. A prefetched result is reused only when a subsequent authoritative query produces a normalized exact match. This verification rule changes when evidence is retrieved without overriding the agent’s authoritative query-generation and reasoning process.

Method

OmniSpec is a tool-layer speculation framework for search-heavy omnimodal agents. It augments an Orchestra-style MainAgent–SubAgent system without modifying the decomposition policy, SubAgent prompts, or answer aggregation, and introduces two mechanisms: *Speculative Reflection* (§), which predicts and asynchronously prefetches likely future queries so that retrieval overlaps with generation, and a *Shared Evidence Pool* (§), which serves authoritative queries from cached evidence under a verify-before-commit contract so that redundant retrieval is eliminated without perturbing the reasoning trajectory.

Problem Formulation

An omnimodal task is $x = (q, \mathcal{M})$, where q is a question and $\mathcal{M}$ denotes auxiliary modality inputs. At each round, the MainAgent delegates subtasks to SubAgents, each following a ReAct-style trajectory $h_u = \langle (z_0, a_0, o_0), \dots, (z_{D_u}, a_{D_u}) \rangle$ with reasoning states z_t , actions a_t , and observations o_t . We focus on retrieval actions $a_t = \text{SEARCH}(s_t)$.

Retrieval depth. Orchestration parallelizes subtasks but not the retrieval *within* them: s_{t+1} is synthesized only after observing the result of s_t . We define the *retrieval depth* of a

trajectory as the length of its longest chain of blocking external searches. Depth determines both exposed latency (each dependent search blocks generation) and billed cost (reformulations of the same information need trigger redundant backend calls; absent cross-round memory, planner restarts can re-issue a near-identical query more than a dozen times, §??). Our objective is to reduce retrieval depth and billed calls while provably preserving what the agent asks and observes.

Why search admits speculation. Unlike code execution or environment-mutating actions, web search is read-only, idempotent, and parallelizable: candidate queries can be issued concurrently and unused results discarded at no correctness cost. Speculation over search therefore needs only to *cover* likely evidence needs, not predict the exact next action.

Speculative Reflection

Speculative Reflection turns the idle time of SubAgent generation into prefetching. After each authoritative search, it (i) grounds the prediction in cross-modal state, (ii) drafts candidate queries with a cheap predictor, and (iii) launches them asynchronously under an explicit budget.

Cross-modal grounding. A recurrent failure mode in omnimodal search is premature generic querying—searching an ambiguous surface form before it is bound to the speaker in an audio clip or the object in a video frame. A rule-based reflection step inspects the current state $c_t = (q, u, h_{<t}, \text{digest}(E), B_t)$ and emits binding signals B_t : which entities to anchor on, whether a numeric question requires a resolved entity first, and which generic query patterns to avoid. Here $\text{digest}(E)$ is a compact (~ 900 -character) summary of the evidence pool.

Drafting and fan-out. Conditioned on (c_t, B_t) , a small off-the-shelf LM (Gemini 2.5 Flash; prompted, not trained) drafts at most k candidate queries targeting evidence still missing—entity disambiguation, source verification, temporal grounding, or cross-modal fact checking. Candidates whose normalized keys already appear in the pool $\mathcal{K}_t$ or the in-flight table $\mathcal{I}_t$ are discarded; survivors are issued concurrently with the SubAgent’s next generation step. Fan-out is bounded by the width k , an in-flight cap ν , and the pool capacity C (defaults $k=\nu=2$, $C=40$), so speculative cost is bounded by construction. Prefetched results never enter the reasoning trace directly; they are only staged in the pool for verified reuse.

Shared Evidence Pool

The pool E is scoped to one task and shared by all SubAgents spawned for it.¹ Each record is

$$e = (\kappa, s, R, m, p, \rho), \quad (1)$$

with normalized key $\kappa=N(s)$ (canonicalized case, whitespace, punctuation), raw query s , result R , modality tags

¹Each clone holding a private pool is the *w/o shared pool* ablation (§??).

m , provenance p , and origin $\rho \in \{\text{SPEC}, \text{AUTH}\}$. All backend results—speculative or authoritative—and all multi-modal extraction outputs are ingested through the same path, content-deduplicated, relevance-scored, and capacity-bounded at C snippets; the pool is thus a unified evidence substrate across modalities, not merely a search cache.

Verify-before-commit reuse. Every authoritative action $a_t = \text{SEARCH}(s_t)$ is resolved through a two-tier gateway before touching the backend. Writing $\kappa_t = N(s_t)$,

$$\text{Obs}(s_t) = \begin{cases} E[\kappa'], & \exists \kappa' \in \mathcal{K}_t: \text{sim}(\kappa_t, \kappa') \geq \tau, \\ \text{Join}(\mathcal{I}_t[\kappa_t]), & \kappa_t \in \mathcal{I}_t, \\ \text{Backend}(s_t), & \text{otherwise.} \end{cases} \quad (2)$$

Tier one is a *dedup hit*: a previously executed query—by any SubAgent, in any round—matching under a token-overlap similarity with threshold τ returns its cached result with zero API cost and zero depth. Tier two is a *prefetch hit*: an in-flight speculation on the same normalized key is joined and committed as authoritative ($\rho \leftarrow \text{AUTH}$). Only on a miss does the backend fire, after which the result is pooled and the next speculative fan-out is triggered without blocking the returned observation (Algorithm ??). Hits are returned in the ordinary backend format, so the SubAgent cannot distinguish reuse from live retrieval.

Scope-aware provisioning. Beyond on-demand reuse, the pool seeds each SubAgent’s working memory with a scope-filtered digest: top-ranked snippets for its subtask, verified entities and modal extractions, and the list of already-issued queries. Provisioning turns the pool from a per-agent cache into a cross-subtask communication channel—an entity resolved from an image by one SubAgent becomes available to its siblings without re-retrieval.

Trajectory preservation. Speculation in OmniSpec is *output-lossless at the query level*: the gateway alters only *when* and *whether* a query hits the backend, never *what* the SubAgent asks or which observation format it receives. Unverified speculative content never enters the reasoning trace; mispredicted prefetches expire with the task. The only relaxation from strict losslessness is the near-duplicate threshold τ , which trades a bounded matching tolerance for higher reuse ($\tau=1$ recovers exact-match reuse).

Cost Model

Let n_{sr} be SubAgent search requests, n_{hit} pool hits (both tiers), and n_{spec} launched speculations with n_{ph} prefetch hits. Billed backend calls are

$$n_{\text{api}} = \underbrace{n_{\text{sr}} - n_{\text{hit}}}_{\text{authoritative}} + \underbrace{n_{\text{spec}} - n_{\text{ph}}}_{\text{speculative misses}}, \quad (3)$$

so OmniSpec saves cost iff $n_{\text{hit}} > n_{\text{spec}} - n_{\text{ph}}$, with the miss term bounded by (k, ν, C) . Latency gains are complementary and accrue even on misses: prefetches run concurrently with generation, a prefetch hit removes a full search round-trip from the critical path, and dedup hits resolve in microseconds.

Orchestra Method	OmniGAIA				Video BrowserComp*				MM-BrowseComp-Acad.			
	Acc. $\uparrow$	Spd. $\uparrow$	Srch. $\downarrow$	Cost $\downarrow$	Acc. $\uparrow$	Spd. $\uparrow$	Srch. $\downarrow$	Cost $\downarrow$	Acc. $\uparrow$	Spd. $\uparrow$	Srch. $\downarrow$	Cost $\downarrow$
<i>MainAgent based on Orchestra-o1-8B</i>												
Orchestra-o1-GS	46.7	1.0 $\times$	1.0 $\times$	1.0 $\times$	49.2	1.0 $\times$	1.0 $\times$	1.0 $\times$	8.3	1.0 $\times$	1.0 $\times$	1.0 $\times$
OmniSpec (Ours)	44.8	1.7 $\times$	0.30 $\times$	0.39 $\times$	51.5	1.7 $\times$	0.09 $\times$	0.26 $\times$	8.3	2.7 $\times$	0.22 $\times$	0.40 $\times$
<i>MainAgent based on GPT-5</i>												
Orchestra-o1-GS	73.1	1.0 $\times$	1.0 $\times$	1.0 $\times$	39.4	1.0 $\times$	1.0 $\times$	1.0 $\times$	21.6	1.0 $\times$	1.0 $\times$	1.0 $\times$
OmniSpec (Ours)	70.3	5.4 $\times$	0.09 $\times$	0.27 $\times$	44.6	3.0 $\times$	0.06 $\times$	0.20 $\times$	20.0	3.9 $\times$	0.43 $\times$	0.48 $\times$

Table 1: Main results under performance–budget trade-offs. **Orchestra-o1-GS** is our reproduced Orchestra baseline using Gemini Grounded Search; *Video BrowserComp is restricted to the 66 questions answerable without an external video-search service. Within each MainAgent group, **Spd.** is the wall-clock speedup T_{GS}/T_{method} ; **Srch.** and **Cost** are the ratios of agent-initiated search calls and total execution cost (inference plus search, \$14/1K queries) relative to the GS baseline. **OmniSpec (Ours)** uses GPT-5 subagents. The 8B group is evaluated on the full set; the GPT-5 group on matched 10-task subsets per benchmark.

Method	Acc. $\uparrow$	Spd. $\uparrow$	Srch. $\downarrow$	Cost $\downarrow$
Orchestra-o1-GS	46.7%	1.00 $\times$	1.00 $\times$	1.00 $\times$
Full OmniSpec	44.8%	1.71 $\times$	0.30 $\times$	0.39 $\times$
w/o Spec. Reflection	42.8%	1.55 $\times$	0.38 $\times$	0.50 $\times$
w/o Shared Evid. Pool	47.2%	1.39 $\times$	0.42 $\times$	0.47 $\times$

Table 2: Accuracy–efficiency analysis of OMNISPEC with the **Orchestra-o1-8B** MainAgent and GPT-5 SubAgents. **Spd.** denotes median wall-clock speedup, **Srch.** denotes the number of billable search calls, including authoritative and speculative searches, and **Cost** includes model-inference and external-tool expenses. All efficiency metrics are normalized to the **Orchestra-o1-GS** baseline. Speculative Reflection primarily suppresses unnecessary retrieval, whereas the Shared Evidence Pool mainly improves execution efficiency, introducing a modest accuracy–efficiency trade-off.

Experiments

Experimental Setup

Benchmarks, framework, and configurations. We primarily evaluate on OmniGAIA, comprising 360 multimodal agentic QA tasks (122 Easy, 160 Medium, 78 Hard across nine domains) that require joint reasoning over video, audio, image, and web evidence. To assess generalization, we further evaluate on VideoBrowserComp, restricted to the 66 questions answerable without an external video-search service—which Orchestra-o1 does not provide—so that all systems share an identical tool stack, and on the Academic subset of MM-BrowseComp, as full paired runs on the complete benchmark are prohibitively expensive. All systems build on Orchestra-o1: a MainAgent decomposes each task into subtasks executed by ReAct-style SubAgents (at most 10 delegation rounds, 30 steps per SubAgent, concurrency 10), with web retrieval via Gemini Grounded Search and all multimodal tools held fixed within every matched pair. We use two configurations: a GPT-5 MainAgent for the principal results (Table 1) and a locally served Orchestra-o1-8B MainAgent for the ablation, retrieval-depth, and fan-out analyses,

each paired with GPT-5 or GPT-4o SubAgents. All efficiency comparisons use our matched Orchestra-o1-GS reproduction, which shares the MainAgent, SubAgent backbone, tool stack, prompts, task set, and execution limits with the corresponding OmniSpec run.

OmniSpec configuration and metrics. OmniSpec is a drop-in tool-layer wrapper over each matched baseline, leaving decomposition, prompts, tool interfaces, and answer aggregation unchanged. The speculative predictor is a prompted `gemini-2.5-flash` with fan-out $k=2$, in-flight cap $\nu=2$, and pool capacity $C=40$ unless otherwise specified; speculation is restricted to read-only web retrieval, each authoritative search is resolved through the verify-before-commit gateway (Eq. 2) with fallback to the original backend, and the Evidence Pool seeds each SubAgent with a scope-filtered digest of evidence and resolved entities. Reproduced runs use a single pass at temperature 0, and we report accuracy as the primary metric, following Orchestra-o1. All efficiency ratios are computed against the matched GS run over the complete charged trace, including speculative misses: **Search** counts all charged backend calls (authoritative and speculative), **Cost** covers model inference, the predictor, multimodal tools, and external search, **Pool Hit Rate** is the fraction of authoritative requests resolved without a new backend call, and **Speedup** is baseline over OmniSpec wall-clock time—throughput speedup under concurrency 10 for end-to-end results, median per-task speedup under concurrency 1 for component and critical-path analyses.

Main Results

Table 1 summarizes the accuracy–efficiency trade-off across three benchmarks and two MainAgent configurations. With the GPT-5 MainAgent on OmniGAIA, OmniSpec attains 70.3% accuracy—within 2.8 points of the matched Orchestra-o1-GS baseline (73.1%)—while reducing agent-initiated search calls to 0.09 $\times$ and total execution cost to 0.27 $\times$, with a 5.4 $\times$ wall-clock speedup. The gains transfer beyond OmniGAIA: on Video BrowserComp, OmniSpec *improves* accuracy from 39.4% to 44.6% while cutting search to

Category	8B MainAgent			GPT-5 MainAgent		
	Base	GS	Ours	Base	GS	Ours
Geo.	21.7	37.7	34.8	72.5	73.9	68.1
Tech.	32.7	42.9	41.7	69.4	67.3	77.6
Hist.	37.9	54.5	53.0	75.8	75.8	77.3
Fin.	12.0	48.0	32.0	64.0	68.0	56.0
Sport	29.7	59.5	48.6	83.8	86.5	73.0
Art	16.7	36.1	50.0	63.9	61.1	50.0
Movie	45.5	51.5	51.5	69.7	66.7	75.8
Sci.	38.5	50.0	50.0	73.1	76.9	69.2
Food	38.9	38.9	44.4	83.3	83.3	77.8
Overall	30.0	46.7	44.8	72.8	73.1	70.3

Table 3: Category-wise accuracy on OmniGAIA. **Base** is the published Orchestra-o1 result, **GS** our reproduction with Gemini Grounded Search, and **Ours** OmniSpec on top of the corresponding GS baseline with a GPT-5 SubAgent. Bold marks the best result within each MainAgent group.

0.06 $\times$ and cost to 0.20 $\times$, and on MM-BrowseComp-Acad. it stays within 1.6 points (21.6% vs. 20.0%) at 0.48 $\times$ cost and a 3.9 $\times$ speedup.

The lightweight Orchestra-o1-8B MainAgent shows the same pattern at a smaller scale: accuracy is preserved within 1.9 points on OmniGAIA (46.7% vs. 44.8%), slightly improved on Video BrowserComp (49.2% vs. 51.5%), and unchanged on MM-BrowseComp-Acad. (8.3%), while search falls to 0.09 $\times$ –0.30 $\times$ and cost to 0.26 $\times$ –0.40 $\times$. Notably, the relative savings are largest under the *stronger* MainAgent: GPT-5 decomposes tasks into more SubAgents and issues far more searches per task, so a larger fraction of its retrieval is redundant or predictable—exactly the volume that speculation and pooled reuse absorb. Speculative evidence orchestration thus becomes more, not less, valuable as orchestration scales.

Table 3 breaks OmniGAIA accuracy down by category. Under both MainAgents, OmniSpec tracks its GS baseline closely across all nine domains and wins several of them outright: with the GPT-5 MainAgent it achieves the best scores on Tech. (77.6%), Hist. (77.3%), and Movie (75.8%), and with the 8B MainAgent on Art (50.0%) and Food (44.4%). The largest drops (e.g., Fin. and Sport under the GPT-5 MainAgent) occur in categories whose questions hinge on fine-grained numeric verification, where occasionally reusing a near-duplicate query in place of a fresh reformulation can miss the exact figure requested; we return to this trade-off in the ablation below.

Analysis

Component Ablation

Table 2 ablates the two components of OmniSpec with the Orchestra-o1-8B MainAgent and GPT-5 SubAgents. Removing Speculative Reflection degrades every efficiency metric: speedup falls from 1.71 $\times$ to 1.55 $\times$, billable search rises from 0.30 $\times$ to 0.38 $\times$, and cost from 0.39 $\times$ to 0.50 $\times$, while accuracy also drops to 42.8%. Without reflection, the

Method	Sub.	Acc.	Depth ↓				Sr. ↓	Cost ↓	Spd. ↑
			E	M	H	All			
MainAgent: Orchestra-o1-8B									
GS	GPT-5	46.7	2	4	5	4	1.0×	1.0×	1.0×
Ours	GPT-5	44.8	1	1.5	2	1	0.30×	0.39×	1.7×
GS	GPT-4o	36.9	2	2	2	2	1.0×	1.0×	1.0×
Ours	GPT-4o	35.4	1	1	1	1	0.40×	0.53×	1.8×
MainAgent: GPT-5									
GS	GPT-5	73.1	18	29	40	27	1.0×	1.0×	1.0×
Ours	GPT-5	70.3	3	3	3	3	0.09×	0.27×	5.4×
GS	GPT-4o	70.6	4	7	12	7	1.0×	1.0×	1.0×
Ours	GPT-4o	71.1	1	1	2	1	0.28×	0.36×	3.7×

Table 4: Critical-path retrieval analysis on OmniGAIA. **Depth** is the median longest chain of blocking external searches per task (**Easy/Med./Hard**; Evidence Pool hits incur zero depth); **Acc.** is accuracy (%); **Sr.**, **Cost**, and **Spd.** denote search calls, total execution cost, and wall-clock speedup, each normalized to the GS baseline with the same SubAgent. Metric definitions are given in the Experimental Setup section.

predictor drafts generic candidates before cross-modal entities are bound, so more speculative calls are billed as misses and the SubAgent still needs additional authoritative reformulations. Removing the Shared Evidence Pool instead attacks reuse directly: each SubAgent keeps only a private cache, cross-step and cross-subtask hits disappear, and speedup collapses to 1.39 $\times$ —the worst of the three variants—even though search volume (0.42 $\times$) remains below baseline thanks to speculation alone.

Accuracy moves within a few points across variants, and the w/o-pool variant even scores highest (47.2%): with fewer cache hits the agent performs more fresh searches, which occasionally surface additional evidence on hard instances, and LLM-judged accuracy over 360 tasks carries non-trivial variance. Crucially, this variant pays roughly 40% more search and 20% more cost for a statistically comparable score; on the accuracy–cost frontier, full OmniSpec dominates both ablations, which is precisely the budget-aware objective the system optimizes.

Critical-Path Retrieval Depth

Table 4 measures the median longest chain of *blocking* external searches per task, reconstructed from execution trajectories. Two baseline patterns stand out. First, depth grows with difficulty (8B MainAgent, GPT-5 SubAgent: 2/4/5 for Easy/Medium/Hard). Second, depth grows sharply with MainAgent strength: the GPT-5 MainAgent spawns more delegation rounds and, lacking cross-round memory, re-issues near-identical queries across retries, driving median depth to 27 overall and 40 on Hard tasks. Sequential retrieval, not reasoning, dominates its critical path.

OmniSpec collapses this path in every configuration: overall median depth falls from 4 to 1 (8B, GPT-5 SubAgent),

Config	Search (avg/task) ↓			Pool Hit Rate (%) ↑		
	Easy	Med.	Hard	Easy	Med.	Hard
GS (no spec.)	7.40	10.03	12.81	—	—	—
$k=1$ (shallow)	1.70	3.02	3.69	24.3	13.9	18.2
$k=2$ (default)	1.76	3.62	4.37	16.3	18.5	26.1
$k=4$ (broad)	3.01	5.33	6.60	12.7	16.9	39.0

Table 5: Speculative fan-out sensitivity (k : fan-out width; in-flight cap matched to k ; GPT-4o SubAgent, Orchestra-o1-8B MainAgent). Search counts authoritative + speculative external calls per task; hit rate is the fraction of authoritative queries served from the pool. GS is the non-speculative baseline.

from 2 to 1 (8B, GPT-4o), from 27 to 3 (GPT-5 MainAgent, GPT-5 SubAgent), and from 7 to 1 (GPT-5 MainAgent, GPT-4o). Because pool hits contribute zero depth, this reduction shows that most dependent searches are moved off the critical path and overlapped with generation. The depth reduction also explains the speedup gradient across configurations: where the baseline chain is deepest (depth 27), speculation hides the most latency ($5.4\times$); where it is shallow (depth 2), the gain is correspondingly modest ($1.8\times$). Accuracy remains within 2.8 points in all four matched pairs—and improves in one (70.6% vs. 71.1%)—confirming that collapsing retrieval depth does not perturb what the agent ultimately asks and observes.

Speculative Fan-Out Sensitivity

Table 5 varies the fan-out width k with the in-flight cap matched to k . Even the shallowest speculation is highly effective: at $k=1$, total external calls drop from 7.40/10.03/12.81 per task (Easy/Medium/Hard, GS baseline) to 1.70/3.02/3.69, a $3\text{--}4\times$ reduction, because pool-based deduplication already removes most redundant reformulations regardless of width.

The optimal width, however, is difficulty-dependent. On Easy tasks, $k=1$ attains the best hit rate (24.3% vs. 16.3% at $k=2$ and 12.7% at $k=4$): easy instances have few distinct evidence needs, so wider fan-out mostly adds billed misses. On Hard tasks the trend reverses monotonically: the hit rate climbs from 18.2% ($k=1$) to 26.1% ($k=2$) and 39.0% ($k=4$), reflecting that difficult omnimodal questions branch over more entities, disambiguations, and verification queries, so broader coverage keeps paying off. The price is visible in the search column: $k=4$ nearly doubles external calls relative to $k=2$ (6.60 vs. 4.37 on Hard), eroding the cost advantage even as latency coverage improves. The default $k=2$ therefore balances the two regimes, and the asymmetry motivates difficulty-adaptive fan-out—shallow speculation for easy instances, broader coverage for hard ones—as a natural extension.

Conclusion

We introduced OMNISPEC, a speculative search and shared-evidence layer for multimodal agentic reasoning. Instead of

changing the orchestrator–subagent interface, OMNISPEC operates at the tool layer, where it anticipates likely search queries, reuses task-wide evidence across modalities, and applies lightweight reflection to improve grounding before retrieval. This design targets a central inefficiency in search-heavy multimodal tasks: agents repeatedly issue overlapping queries and wait on external tools even when much of the required evidence is predictable or reusable. By overlapping retrieval with generation and consolidating evidence within each task, OMNISPEC substantially reduces redundant search and tool use while preserving the behavior of the original reasoning pipeline. Empirically, it achieves comparable accuracy to the unmodified ORCHESTRA-O1 system while improving cost and latency, suggesting that much of the overhead in multimodal agentic reasoning can be addressed without stronger models or changes to the agent contract. More broadly, our results point to speculative execution and evidence reuse as practical system-level mechanisms for making multimodal agents more efficient. Future work will explore adaptive speculation policies, broader support for multimodal extraction tools, and more conservative reuse strategies for difficult video-centric tasks where additional exploration remains important.

References

- Chen, C.; Borgeaud, S.; Irving, G.; Lespiau, J.-B.; Sifre, L.; and Jumper, J. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.
- Choi, D.; Park, K.; Song, W.; Dingliwal, S.; Jayanthi, S. M.; Shin, J.; and Galstyan, A. 2026. IdleSpec: Exploiting Idle Time via Speculative Planning for LLM Agents. *arXiv preprint arXiv:2605.22154*.
- Guan, Y.; Lan, Q.; Fei, S.; Ding, D.; Acharya, D.; Wang, C.; Wang, W. Y.; and Hua, W. 2025. Dynamic speculative agent planning. *arXiv preprint arXiv:2509.01920*.
- Hooper, C.; Kang, M.; Moon, S.; Lee, N.; Wen, E.; Wawrzyniec, J.; Mahoney, M. W.; Shao, Y. S.; Gholami, A.; and Keutzer, K. 2026. Speculative Interaction Agents: Building Real-Time Agents with Asynchronous I/O and Speculative Tool Calling. *arXiv e-prints*, arXiv–2605.
- Huang, H.; Huang, J.; Wan, Z.; Zheng, X.; Ji, R.; and Luo, J. 2026. SpecEyes: Accelerating agentic multimodal LLMs via speculative perception and planning. *arXiv preprint arXiv:2603.23483*.
- Leviathan, Y.; Kalman, M.; and Matias, Y. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 19274–19286. PMLR.
- Li, X.; Jiao, W.; Jin, J.; Wang, S.; Dong, G.; Jin, J.; Wang, H.; Wang, Y.; Wen, J.-R.; Lu, Y.; et al. 2026. Omni-gaia: Towards native omni-modal ai agents. *arXiv preprint arXiv:2602.22897*.
- Lin, H.; Shi, Y.; Geng, T.; Zhao, W.; Wang, W.; and Singh, R. P. 2025. Agent-Omni: Test-Time Multimodal Reasoning via Model Coordination for Understanding Anything. *arXiv preprint arXiv:2511.02834*.

Nichols, D.; Singhanian, P.; Jekel, C.; Bhatele, A.; and Menon, H. 2025. Optimizing Agentic Language Model Inference via Speculative Tool Calls. *arXiv preprint arXiv:2512.15834*.

Sui, Y.; Zhao, H.; Ma, R.; He, Z.; Wang, H.; Li, J.; Xu, K.; Chen, K.; and Yang, Y. 2026. Parallelizing Tool Execution and LLM Generation for Low-Latency Agent Serving. *arXiv preprint arXiv:2603.18897*.

Tao, K.; Du, W.; Yu, B.; Wang, W.; Liu, J.; and Wang, H. 2025. OmniAgent: Audio-Guided Active Perception Agent for Omnimodal Audio-Video Understanding. *arXiv preprint arXiv:2512.23646*.

Xing, Z.; Xu, R.; Wang, Y.; He, J.; Ma, Z.; Yang, Q.; Chu, Y.; Xu, J.; Lin, J.; Fu, C.-W.; et al. 2026. Native Active Perception as Reasoning for Omni-Modal Understanding. *arXiv preprint arXiv:2606.19341*.

Ye, N.; Ahuja, A.; Liargkovas, G.; Lu, Y.; Kaffes, K.; and Peng, T. 2025. Speculative Actions: A Lossless Framework for Faster Agentic Systems. *arXiv preprint arXiv:2510.04371*.

Zhang, F.; Zhang, V.; Qian, S.; Li, H.; Wu, H.; Wu, J.; Zhou, D.; Zhu, Z.; Lian, Z.; Wang, X.; et al. 2026. Orchestra-o1: Omnimodal Agent Orchestration. *arXiv preprint arXiv:2606.13707*.

Zhang, W.; and Pei, S. 2026. Predictive Prefetching for Retrieval-Augmented Generation. *Proceedings of Machine Learning Research*.

Zhu, Y.; Mu, X.; Feng, T.; Ou, Z.; Gong, Y.; and Luo, H. 2026. OmniRAG-Agent: Agentic Omnimodal Reasoning for Low-Resource Long Audio-Video Question Answering. *arXiv preprint arXiv:2602.03707*.